Dictionary in Place of Match Case

Replace long if/elif or match chains with a lookup table

The problem: a long chain

```
def day_type(day):
    if day == "sat" or day == "sun":
        return "weekend"
    elif day == "fri":
        return "almost weekend"
    elif day in ("mon", "tue", "wed", "thu"):
        return "weekday"
    else:
        return "unknown"
```

Dictionary lookup

Keys are the cases, values are the results. Add a case by adding a line of data.

```
DAY_TYPE = {
    "mon": "weekday", "tue": "weekday",
    "wed": "weekday", "thu": "weekday",
    "fri": "almost weekend",
    "sat": "weekend", "sun": "weekend",
}
print(DAY_TYPE.get("sat", "unknown"))
print(DAY_TYPE.get("xyz", "unknown"))
```

OUTPUT

```
weekend
unknown
```

The default case

Style	Missing key gives
`d[key]`	`KeyError`
`d.get(key)`	`None`
`d.get(key, default)`	your default: the `case _` equivalent
`if key in d:`	explicit test first

Equivalent match / case

```
match day:
    case "sat" | "sun":
        r = "weekend"
    case "fri":
        r = "almost weekend"
    case "mon" | "tue" | "wed" | "thu":
        r = "weekday"
    case _:
        r = "unknown"
```

Dispatch table: store functions

Map a command to the function to call. Store the function, **not** its call: no parentheses in the dict.

```
def add(a, b): return a + b
def sub(a, b): return a - b
OPS = {"+": add, "-": sub,
       "*": lambda a, b: a * b}
def calc(a, op, b):
    func = OPS.get(op)
    if func is None:
        return "bad operator"
    return func(a, b)
print(calc(6, "+", 3), calc(6, "*", 3), calc(6, "/", 3))
```

OUTPUT

```
9 18 bad operator
```

Menu with actions

```
def start(): return "Starting"
def stop():  return "Stopping"
MENU = {"1": start, "2": stop}
choice = "2"
action = MENU.get(choice, lambda: "Invalid")
print(action())
```

OUTPUT

```
Stopping
```

Group many keys with one value

```
RATE = {}
for keys, v in ((("A", "B"), 10), (("C",), 20)):
    for k in keys:
        RATE[k] = v
print(RATE)
```

OUTPUT

```
{'A': 10, 'B': 10, 'C': 20}
```

Dict or match?

Pick a dict when	Pick match / if when
cases are fixed values	ranges (`score >= 90`)
result is a value or function	you test shape or type
table is data you may edit or load	guards or several conditions
many cases, same code path	branches run different statements

REMEMBER

- Use `d.get(key, default)` to mimic `case _:`; use `d[key]` when a miss should be an error.
- Keys must be hashable (str, int, tuple), and must **equal** the value being tested.
- A dict cannot test ranges or conditions: keep `if/elif` or `match` for those.
- Store functions without parentheses; call the one you pick: `OPS[op](a, b)`.
- Cases become data, so you can load them from JSON or a config file.
- Dict lookup is O(1) on average, so it stays fast however many cases you add.